The State of Context Engineering

Drew Breunig

2026-10-24

The Job is Knowing When to Be Deterministic & When to Be Probabilistic

This week, I gave a short talk at AGNTCon titled, "Context Fails & Fixes," that functioned as a sort of "State of Context Engineering." How did context engineering emerge, where has it gone, and where might it go in the future? As model makers target new capabilities, they inadvertently create challenges we, as users and harness builders, must overcome.

Below are the slides and a lightly edited transcript. As always, shoot me a note with any feedback, thoughts, or questions. I really appreciate it.



Figure 1: Title slide: Context Fails & Fixes. The continuing challenge of preparing precise instructions. Drew Breunig, Cmpnd, October 2026.

Hi, everyone. Thanks for joining me today. We're going to walk through context fails and fixes.

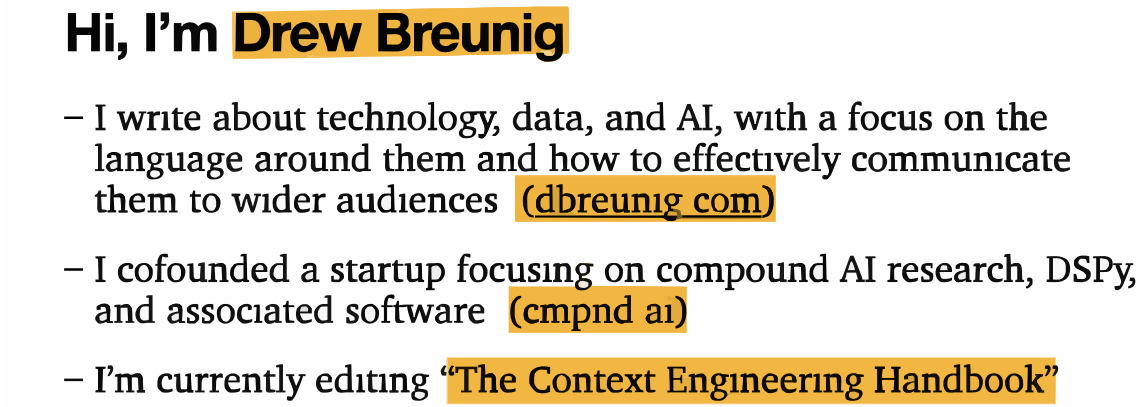


Figure 2: Hi, I'm Drew Breunig. I write about technology, data, and AI at dbreunig.com; I cofounded Cmpnd, a startup focused on compound AI research and DSPy; and I'm currently editing The Context Engineering Handbook.

I have a book coming out in a bit, *The Context Engineering Handbook*. When I first talked to the team at O'Reilly, back in June 2025, they asked me a very reasonable question: "How do we know this won't become obsolete in a few weeks?" The trouble with AI textbooks is that many age extraordinarily quickly.

I said, I don't know, but I'm optimistic. Giving clear instructions about what to do and what not to do, and ensuring those instructions are consistent and not noisy, will forever be a challenge with AI, no matter how good it gets.

We have coworkers, partners, and other people we know and interact with who *are* sentient. They pass the Turing test, they are "AGI," and yet: we still have a hard time conveying instructions, ideas, and concepts among each other.

Context engineering will continue to be hard. But it will be hard in unique ways, because models aren't people. You can make assumptions about how models think or interpret things (use whatever anthropomorphizing word you like), but it helps to understand how they're built, optimized, and trained. Their unique attributes and construction create a unique set of failure modes we need to reckon with.

So let's review how we got here.

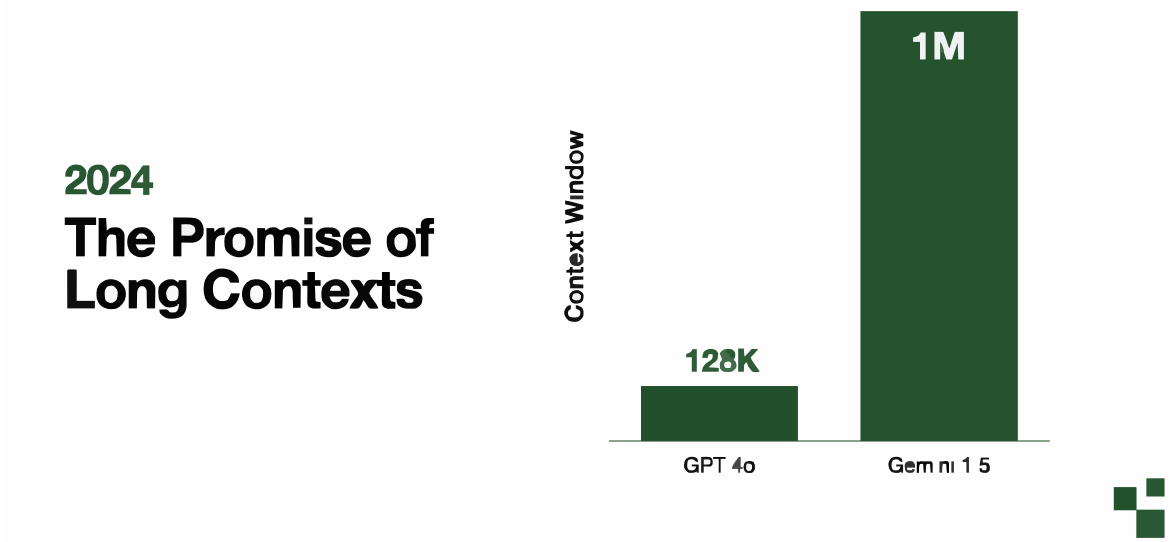


Figure 3: 2024: The Promise of Long Contexts. Bar chart of context window sizes: GPT-4o at 128K tokens versus Gemini 1.5 at 1M.

Let's flash back to the end of 2024.

If you remember, this was the height of the "RAG is dead" moment.

The hope was: if we can just make the context window large enough, we can put the entire internet in it. We can put everything in it. We don't have to be selective. We don't have to build agentic systems. We can fit it all in there, and AI will be solved.

Model builders started aiming at long context capabilities and we had a big leap from ~128,000-token context window models to million-token context window models, starting with Gemini 1.5.

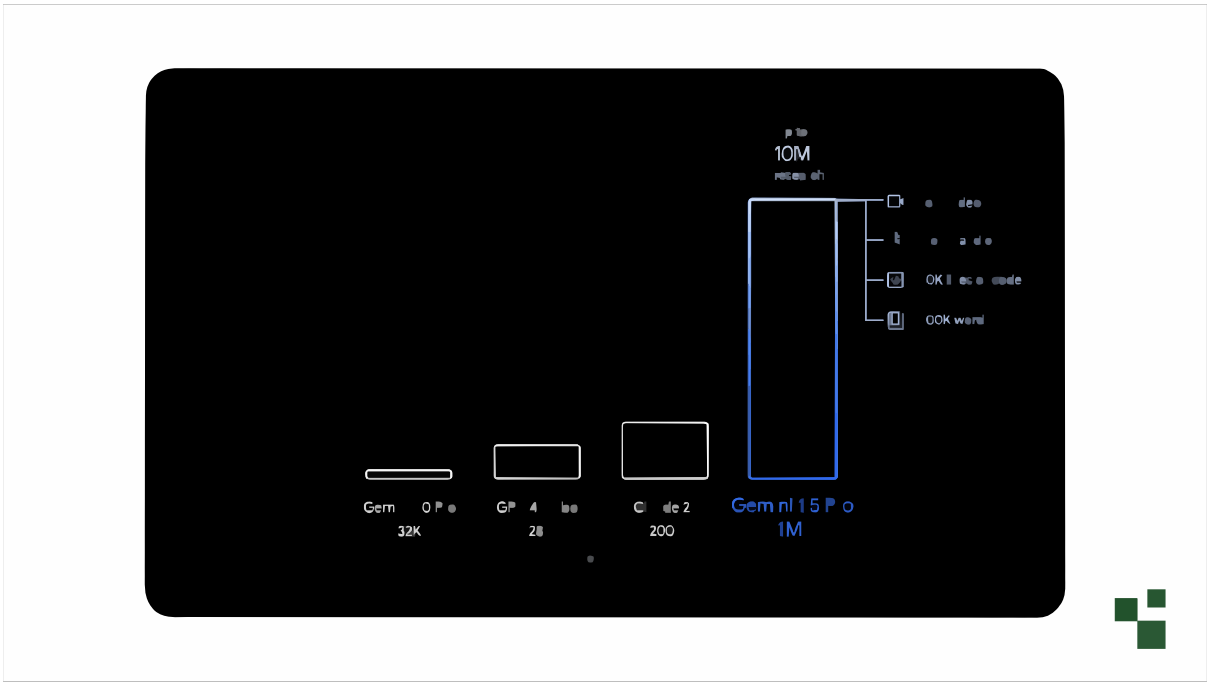


Figure 4: Google's Gemini 1.5 Pro marketing graphic comparing context windows: Gemini 1.0 Pro at 32K, GPT-4 Turbo at 128K, Claude 2.1 at 200K, and Gemini 1.5 Pro at 1M, with up to 10M in research, equal to 1 hour of video, 11 hours of audio, over 30K lines of code, or over 700K words.

If you look at the marketing material for Gemini 1.5 Pro, it was mostly about the million-token context window. They were even teasing a *10-million-token context window* model they had in the lab, that would come out soon.

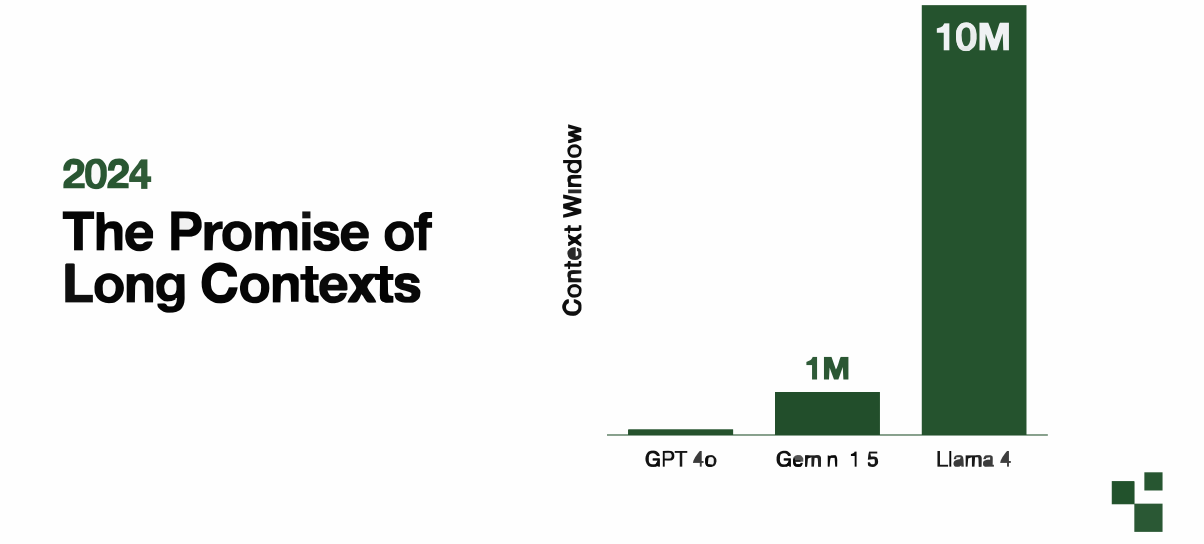


Figure 5: 2024: The Promise of Long Contexts. Bar chart of context windows: GPT-4o, Gemini 1.5 at 1M, and Llama 4 at 10M tokens.

But they got beaten to the punch. Llama 4 Scout came out with a *10-million-token context window*, and well, it wasn't quite what we hoped. Inference was a challenge, and Scout never really caught on, and we never got another Llama.

But we still got a million tokens. That's pretty good. We'll take it.

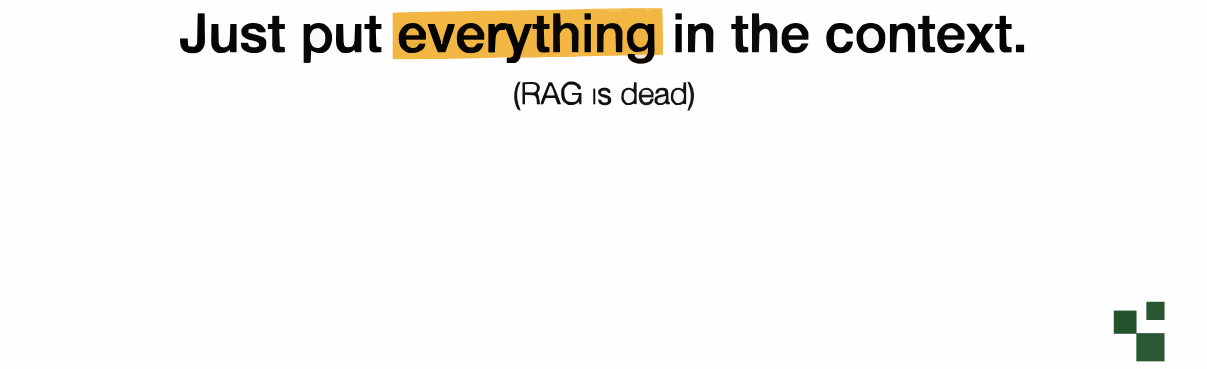


Figure 6: Just put everything in the context. (RAG is dead)

So this was the brief at the time: just put everything in the context and let the model sort it out.

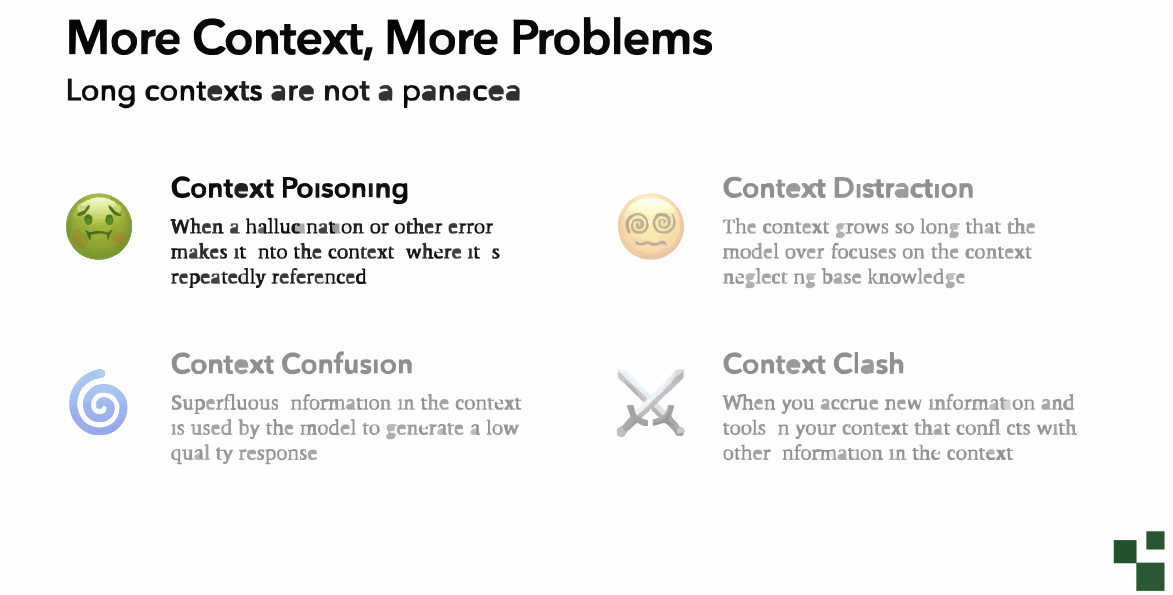


Figure 7: More Context, More Problems: long contexts are not a panacea. Four failure modes: Context Poisoning, when a hallucination or error is repeatedly referenced; Context Distraction, when the model over-focuses on a long context and neglects base knowledge; Context Confusion, when superfluous information produces low-quality responses; and Context Clash, when new information and tools conflict with other information in the context.

But with more context, we had room for more problems. Long contexts are not a panacea.

When I wrote about this, I identified four key failure modes.

Context poisoning is when something gets into the context that shouldn't be there. It could be a hallucination or a bad fact, and the model can't get past it.

Context distraction is when the context grows so long that the model starts to focus on the context over its base knowledge. We can see this in some really interesting ways.

Context confusion is when extraneous facts—things that don't relate to the problem I'm working on—come in and influence the model in ways we don't fully understand. This one's really pernicious. We're not going to dig into it today, but it's one of my favorite failure modes. The model makes assumptions about who you are, and those assumptions influence the answer.

Finally, **context clash** is when tool calls, skills, and other information within your harness start to conflict and work against you in ways you didn't expect.

(How many of you keep memory on in ChatGPT or Claude? I turn it off. I hate it. Sometimes I'm talking about work conversations; sometimes I'm talking about a project I'm doing with my son. The model will take notes on both, mix up those contexts, and the confusion grows from there.)



Figure 8: Box art for Pokémon Red Version on the Game Boy, 1996, with a lemonade icon, next to Pokémon FireRed Version on the Game Boy Advance, 2004, with a tea icon.

Let me give you my favorite example of context poisoning.

I'm going to date myself: I played the game on the left. The one from 1996, for the original Game Boy. This is the first *Pokémon Red*. It was in black and white.

In 2004, Nintendo released *Pokémon FireRed* for the Game Boy Advance. These were essentially the same game, except one had better colors and audio.

But there was one subtle change. At some point in both Pokémon Reds, to get past a security guard, you have to give them something to drink. A bribe. Quite the practical lesson for children.

In the original game, you need to give the thirsty guard lemonade. In *FireRed*, for some reason, they changed it to iced tea.

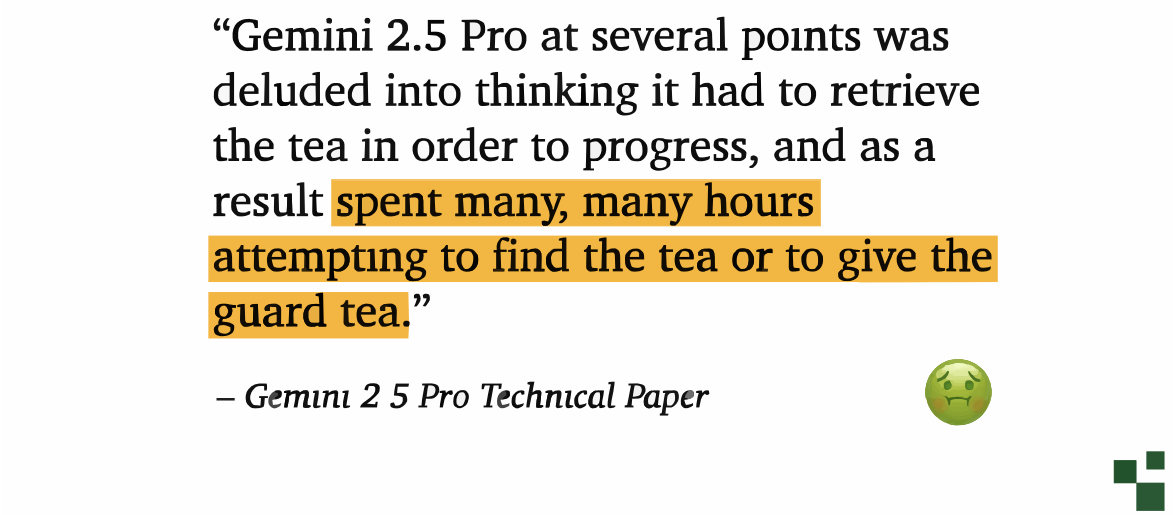


Figure 9: Quote from the Gemini 2.5 Pro Technical Paper: Gemini 2.5 Pro at several points was deluded into thinking it had to retrieve the tea in order to progress, and as a result spent many, many hours attempting to find the tea or to give the guard tea. Marked with a sick-face emoji for context poisoning.

That created a problem for the agent. In the Gemini 2.5 technical report, the researchers described how they had the model play Pokémon. Researchers like Pokémon as an agent benchmark because it's a reasonably controlled environment and doesn't move particularly fast.

Several times, Gemini 2.5 became convinced it needed to get tea, even though it was playing the older game. The tea task for *FireRed* was in its weights, so it poisoned its context. It spent hours trying to find or give the guard tea. Once it had been added to its to-do list, the model could not get past it.

That's context poisoning. And it happens all the time.

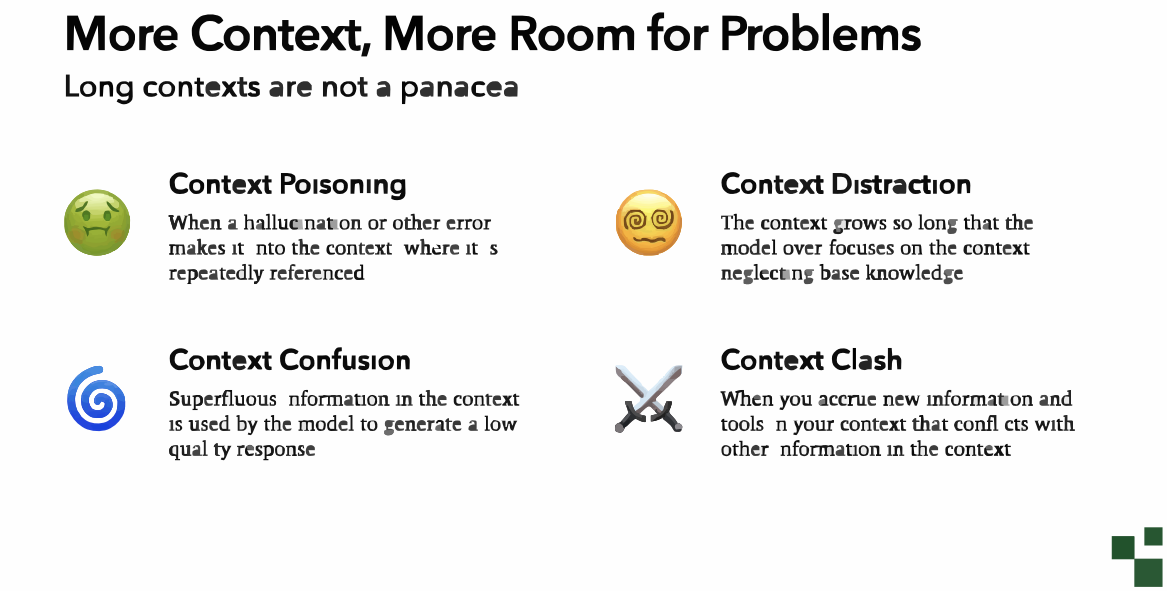


Figure 10: More Context, More Room for Problems: the four context failure modes, with Context Distraction highlighted. The context grows so long that the model over-focuses on the context, neglecting base knowledge.

Another failure mode is context distraction. This is one of my favorite demonstrations, and the research came out after I wrote the original piece.

As your context gets longer, how does your behavior change? Quick question: how many of you, when you're coding, let the context go all the way to a million tokens? Fill it all up?

(About half the room raised their hands!)

Wow, that's more than I expected.

For me, around 200,000 or 300,000 is where I start looking. I'll check, see it's at 250,000, and think: I've got a big job coming up. I'm going to reset and start fresh. Let me show you why this matters.

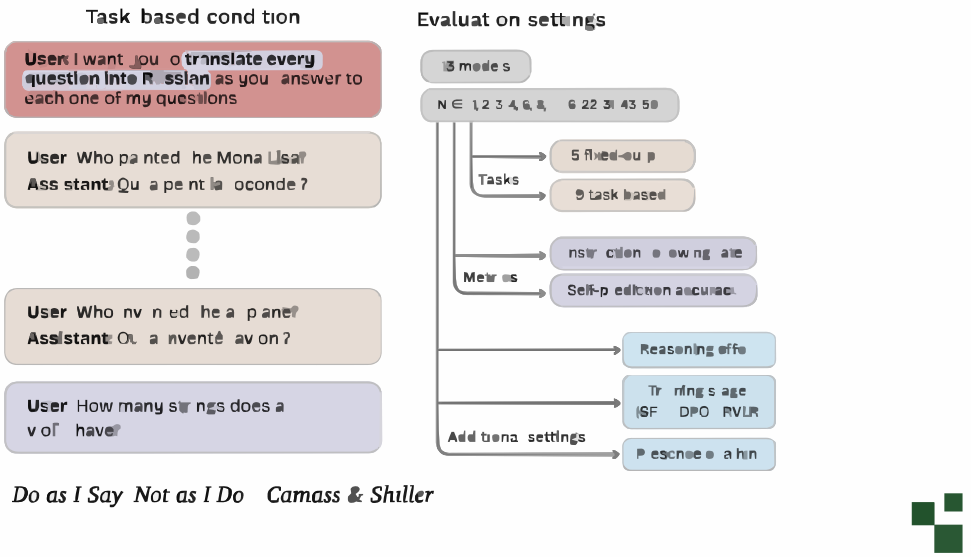


Figure 11: Diagram from Do as I Say, Not as I Do by Camassa and Shiller. The user instructs the model to translate every question into Russian, followed by example turns where the assistant answers in French, then asks how many strings a violin has. Evaluation settings cover 13 models, varying numbers of example turns, instruction-following rate, and reasoning effort.

Recently, I stumbled on a paper that wonderfully illustrates context distraction: *Do as I Say, Not as I Do*, by Carolina Camassa and Derek Shiller.

Their experiment is a little contrived, but it's useful for thinking about how this happens. They give the model an instruction: "I want you to translate every question into Russian as your answer to each of my questions." Then they present a bunch of examples that are counter to these instructions. In this case, many questions with answers in French.

These aren't new instructions. They're just examples sitting in the chat history.

Then they ask the model a question. Will it follow the instructions?

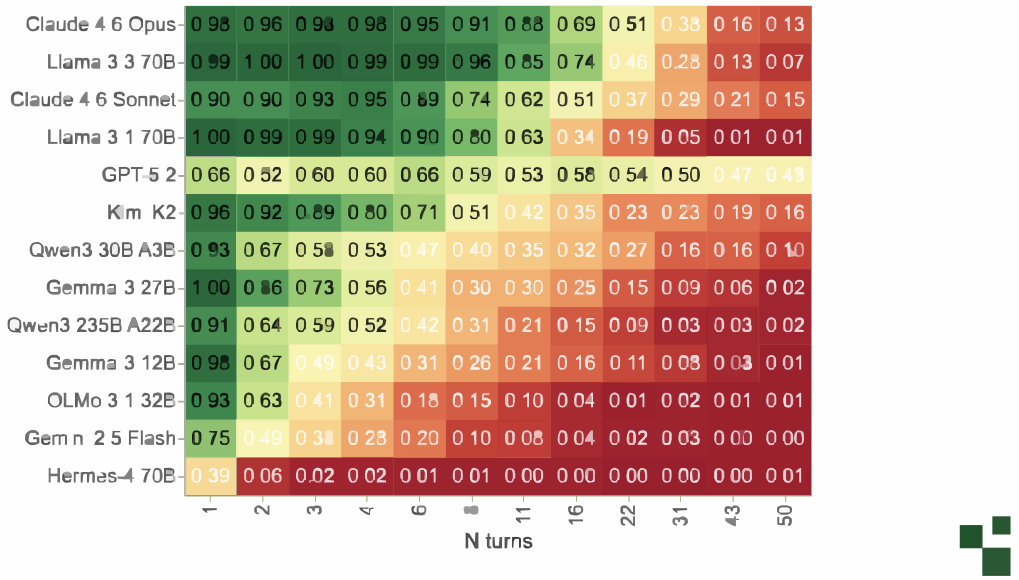


Figure 12: Heatmap of instruction-following rates across models as the number of contradictory example turns grows from 1 to 50. Strong models like Claude 4.6 Opus stay green longer, while smaller models like Gemma 3 12B, OLMo 3.1 32B, and Hermes 4 70B drop to red after only a few turns.

How often does the model respond in Russian, as it was instructed to, and how often does it respond in French, as it's seen in the examples?

The x-axis on this matrix enumerates how many bad examples were present. The y-axis is the model being tested. The figures in the intersections are the percentage of times the model adheres to the instructions, rather than following the examples.

It turns out this affects every model! For smaller models, like Gemma 3 12B, after just a few contradictory examples, the model follows the instruction less than half the time. With Opus 4.6, which is an amazing model, it takes many more examples, but it happens there too.

Imagine you're going back and forth on a front-end design, trying to tweak CSS and get a component exactly right. How many bad examples have you seeded in that exchange? *Those examples start to influence the model.* As your context grows, things fall apart.

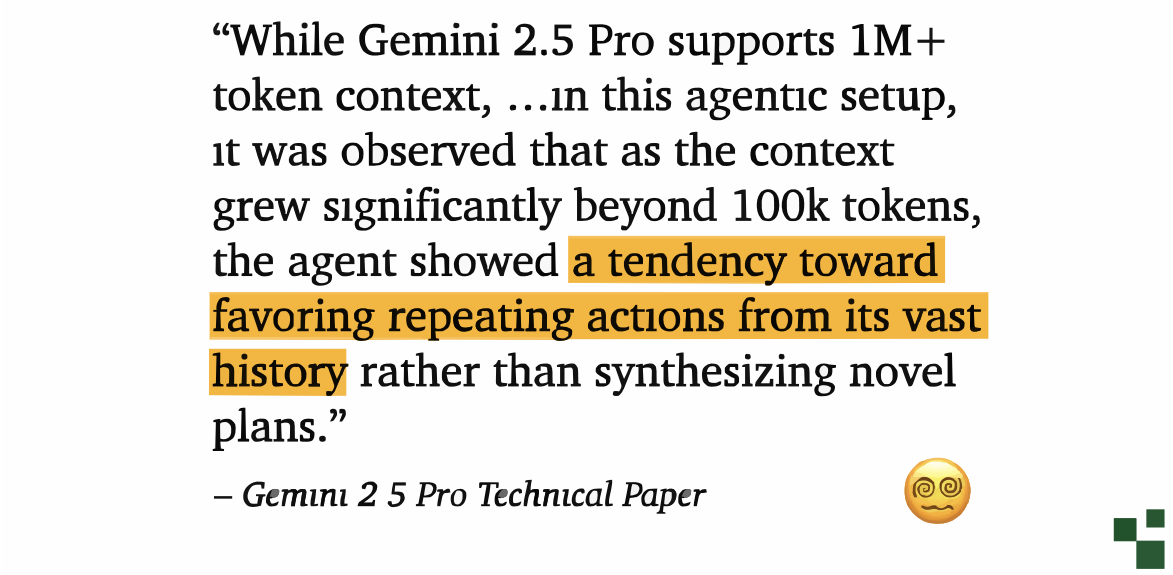


Figure 13: Quote from the Gemini 2.5 Pro Technical Paper: While Gemini 2.5 Pro supports 1M+ token context, in this agentic setup, it was observed that as the context grew significantly beyond 100k tokens, the agent showed a tendency toward favoring repeating actions from its vast history rather than synthesizing novel plans.

Gemini 2.5 ran into this too, while playing Pokémon.

Remember, Google was really proud of its million-token context window. The paper suggested that two million tokens were around the corner. But buried in the appendix was this observation: once the context grew significantly beyond 100,000 tokens, the agent began repeating actions from its history instead of synthesizing new plans.

I'm convinced the marketing people never read the appendix. I love this technical report.

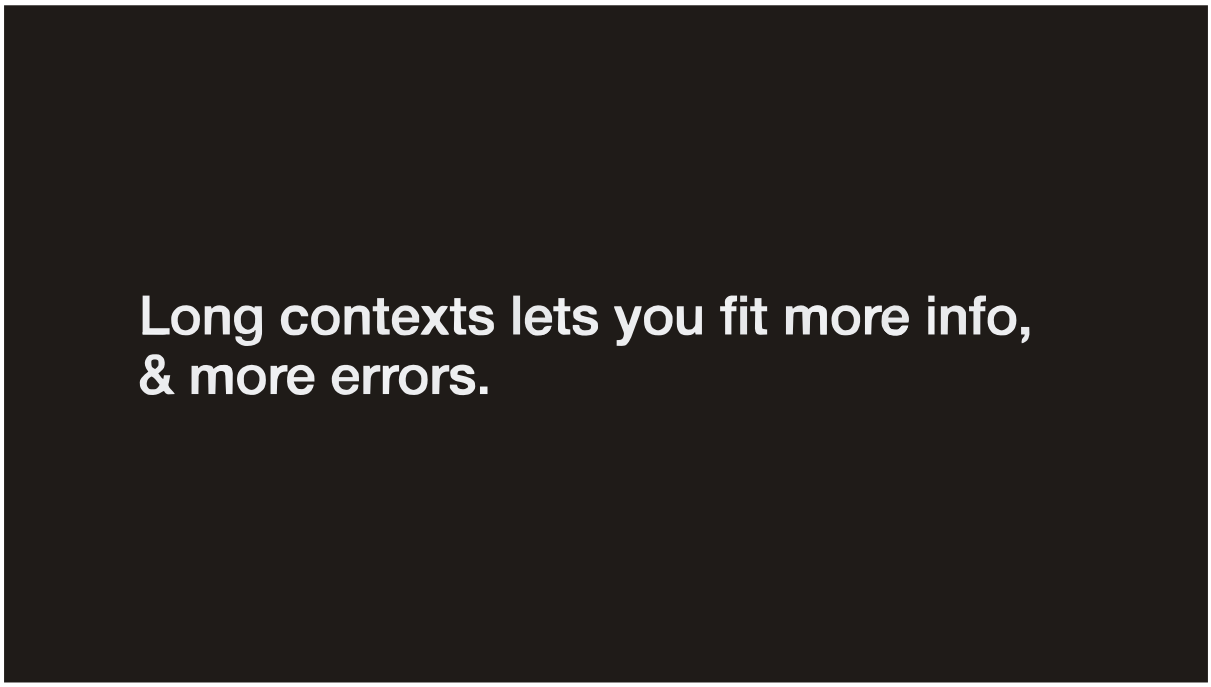


Figure 14: Long contexts lets you fit more info, & more errors.

Long contexts let you fit in more information, but they also let you fit in more errors. Long contexts weren't a panacea.

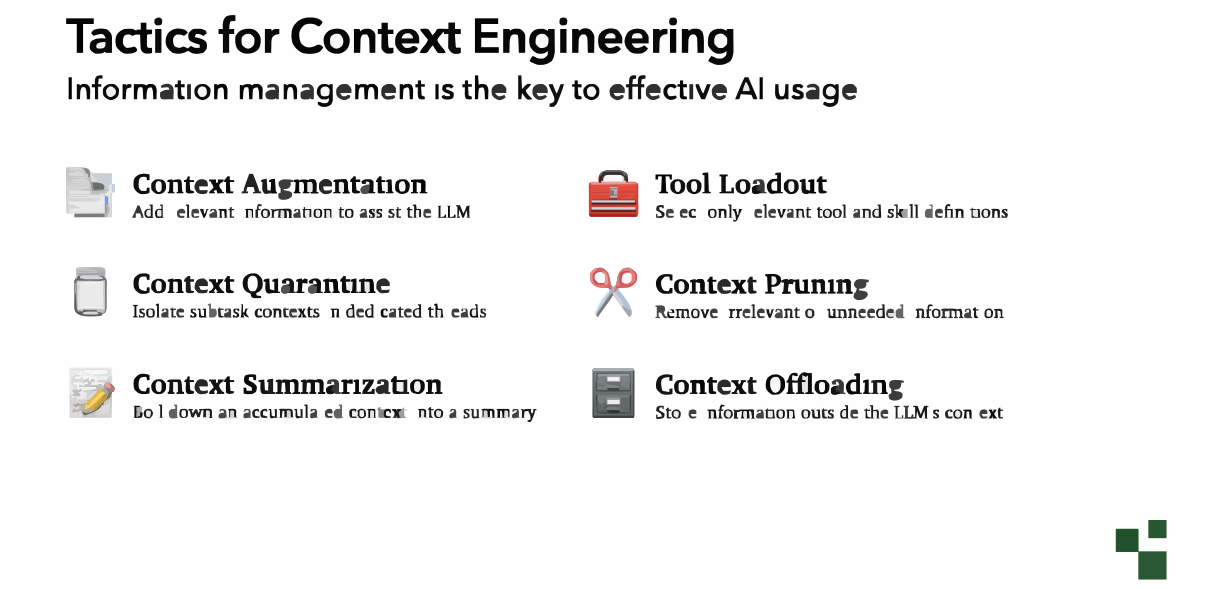


Figure 15: Tactics for Context Engineering: information management is the key to effective AI usage. Six tactics: Context Augmentation, Tool Loadout, Context Quarantine, Context Pruning, Context Summarization, and Context Offloading.

Thankfully, we've developed a pretty good toolbox. Surprisingly, even as new challenges have emerged (we'll get to those in a minute), this set of tools has remained the same.

These six of them.

You have **context augmentation**: RAG, skills, and other ways to bring in the information relevant to the problem.

You have **tool loadout**: selecting which tools and skills to expose so you don't overload the model. I was recently talking to an enterprise team whose agents had suddenly started performing terribly. We investigated and discovered their enterprise software was loading 600 skills for every engineer at startup. We figured they should probably filter those, with a loadout function.

Third is **context quarantine**: subagents and peer agents, where you start something with a blank history but let it work in the same environment.

Fourth is **context pruning**. Claude Code and Codex do this, sometimes without you even noticing. Early stages of compaction often strip out tool results you no longer need.

Fifth is **context offloading**: storing information in the environment, such as Markdown files or a database, so you can access it later.

And sixth is **context summarization**, which everyone knows as compaction.

These are the tools, and we've increasingly moved them into the harness.

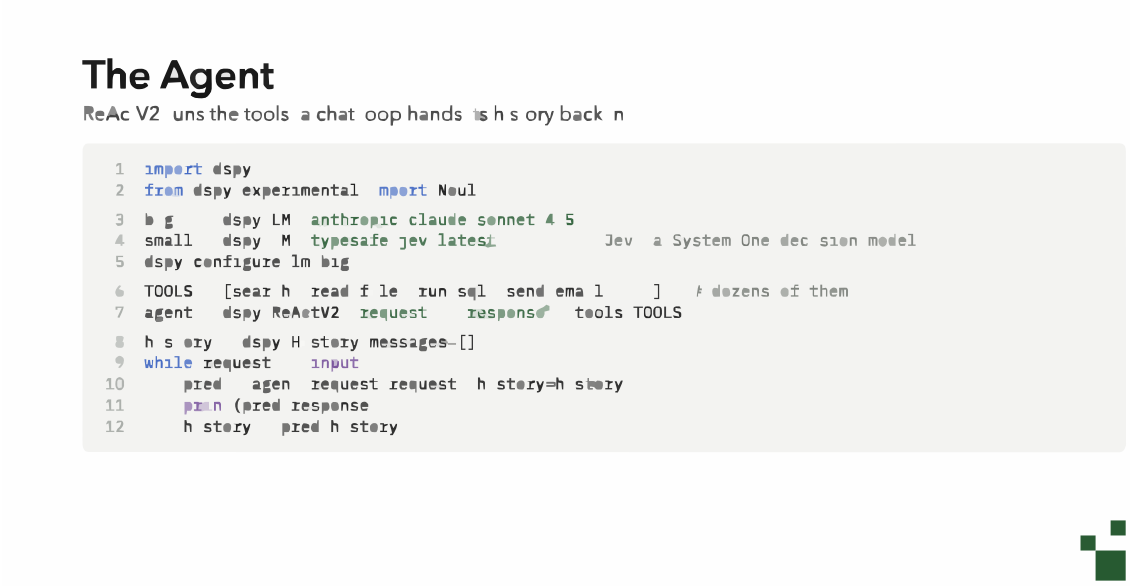


Figure 16: The Agent: DSPy code for a basic agent. It configures a big model, Claude Sonnet 4.5, and a small Jev decision model, defines a list of tools, creates a ReAct agent, and runs a while loop that passes the chat history back in each turn. About a dozen lines.

Let's look at an example harness, to show how simple these patterns are.

I'm really passionate about DSPy. I think it's an incredibly readable and effective way to build AI workflows and agents. And it's also a great language for simply specifying behavior, so it's well suited to this demonstration.

An agent is a `while` loop. Here we're using a ReAct module: we give it a request, we want a response, and we give it some tools. Those tools are just Python functions.

For this demo we have a big model (in this case, Claude opted for Claude Sonnet 4.5, good luck with that!). And I'm using a Jev as the small model: a tiny decision model that makes lightning-fast decisions. You could swap in a small LLM in this example, and everything would still work.

We have a `while` loop that takes input, sends it to ReAct, and carries the accruing history forward. This is an agent in about a dozen lines using DSPy. It would be even fewer if you took out the extra model.

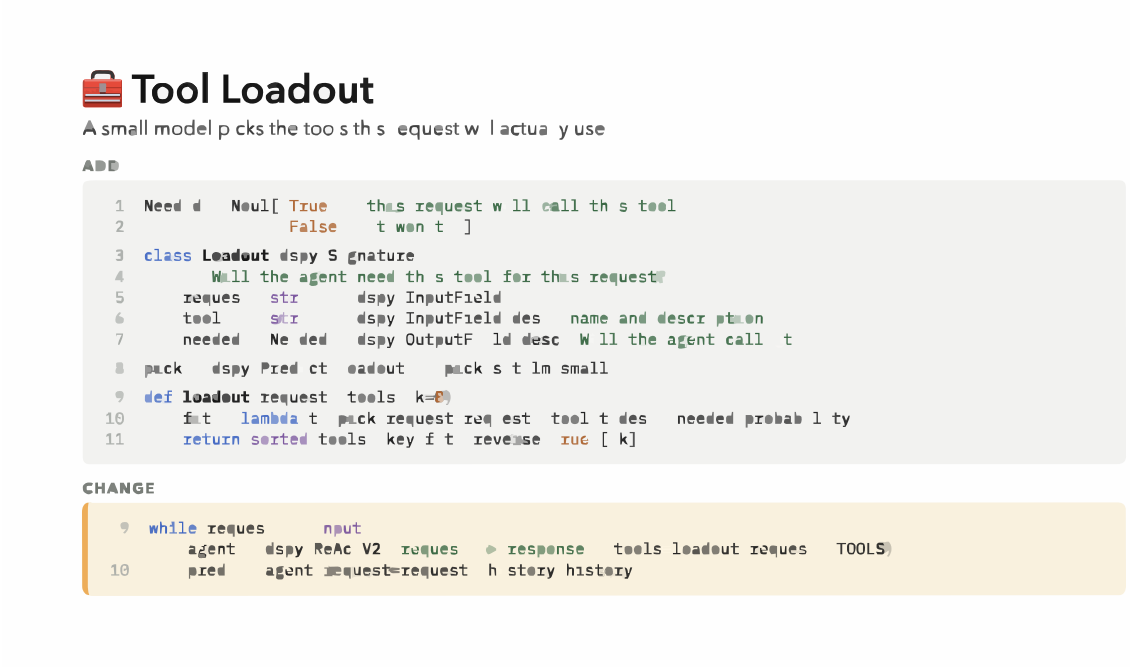


Figure 17: Tool Loadout: DSPy code where a small model picks the tools a request will actually use. A Loadout signature asks whether the agent needs a given tool for a request, and a loadout function ranks the tools and keeps the top few. The agent loop is changed to build its ReAct agent with only the selected tools.

How do we do tool loadout? It's just a function.

We present Jev with the request and a tool, asking if it's needed for this task.

Jev is fast enough that we can do this one tool at a time; it'll be fine. We wrap this in a `loadout` function that iterates through all the tools and asks if each is needed, then returns the top-ranking subset. That's it.

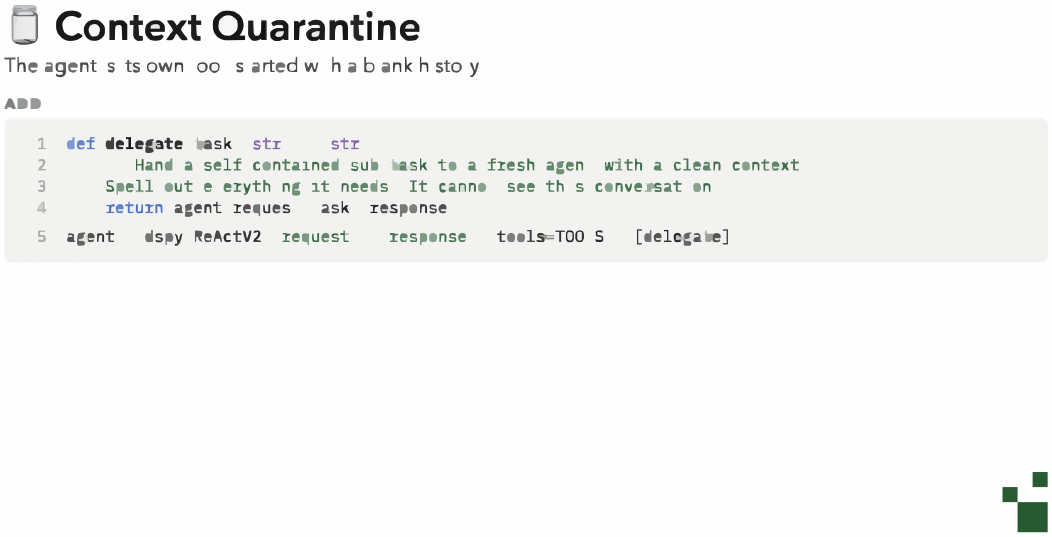


Figure 18: Context Quarantine: DSPy code for a delegate function that hands a self-contained subtask to a fresh agent with a clean context, then adds delegate to the main agent's tools.

With context quarantine, it's just a subagent. You expose an agent as a tool that the main agent can use, and it starts with a blank history.

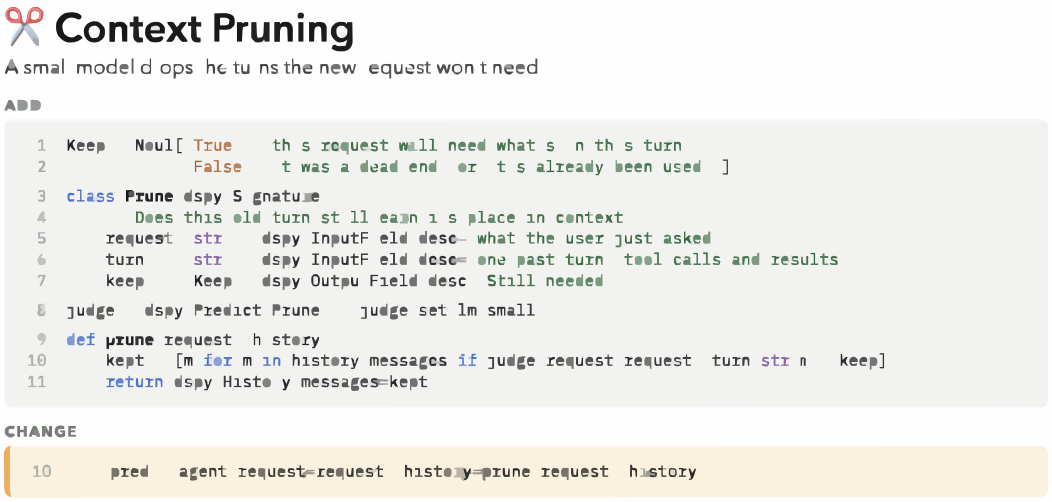


Figure 19: Context Pruning: DSPy code where a small model drops history turns the new request won't need. A Prune signature asks whether each past turn still earns its place in context, and the agent is called with the pruned history.

With context pruning, we're stepping through the history, passing each turn to Jev, and asking: is

this still needed? If not, cut it.

You can be as rough or as elegant as you want here. Just mind your prompt caching.



Figure 20: Context Summarization: DSPy code that rewrites the history into one briefing. A Rewrite signature folds past turns into a briefing a fresh agent can pick up cold, and when the history exceeds 20 messages the loop replaces it with the compacted briefing.

We can also rewrite the history with compaction. Take the old turns, create a summary, and start the new context from that summary.

One of my favorite variations lately is checking what matters *before* you compact. You use Jev, or another decision model, for the first pass to judge what's relevant. Now you've got a smaller subset of the history, and you summarize that. It's a simple extra step, but it makes the summary more useful.



Figure 21: All Together: four fixes, one loop, four lines changed. The agent loop with quarantine, loadout, pruning, and summarization each added with a single line.

Put it all together and it's about 16 lines of changes. Four fixes, one loop. These are the moves we have in the harness. It's actually remarkably simple.

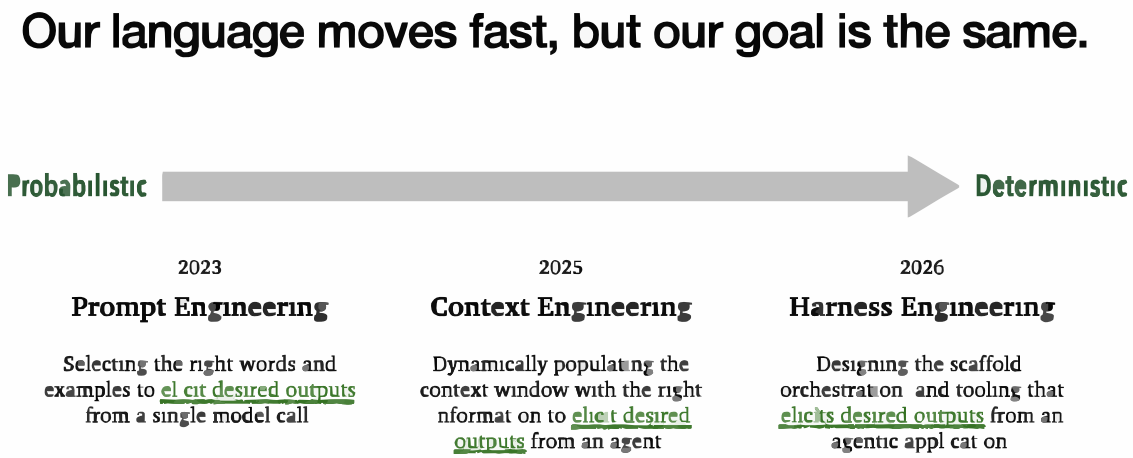


Figure 22: Our language moves fast, but our goal is the same. A timeline from probabilistic to deterministic: 2023 Prompt Engineering, selecting the right words and examples to elicit desired outputs from a single model call; 2025 Context Engineering, dynamically populating the context window with the right information; 2026 Harness Engineering, designing the scaffold, orchestration, and tooling that elicits desired outputs from an agentic application.

What isn't simple is our language: how we talk about this stuff. It's really hard to keep pace; it changes every day.

One of my favorite quotes is from Stewart Brand, which I think about all the time:

> If you want to know where the future is being made, look for where language is being invented and lawyers are congregating.

Certainly applies here! But as our language has changed, our goal has remained the same.

In 2023, we called choosing the right information to get the right response **prompt engineering**.

In 2025, we called it **context engineering**, except that now it was more automated. It wasn't just you writing the prompt; it was your system dynamically assembling the context.

Then, as we moved more of these tools out of the context and into code, we started calling it **harness engineering**.

But the job is the same. We're moving more and more of the work of assembling context from probabilistic model calls to deterministic code.

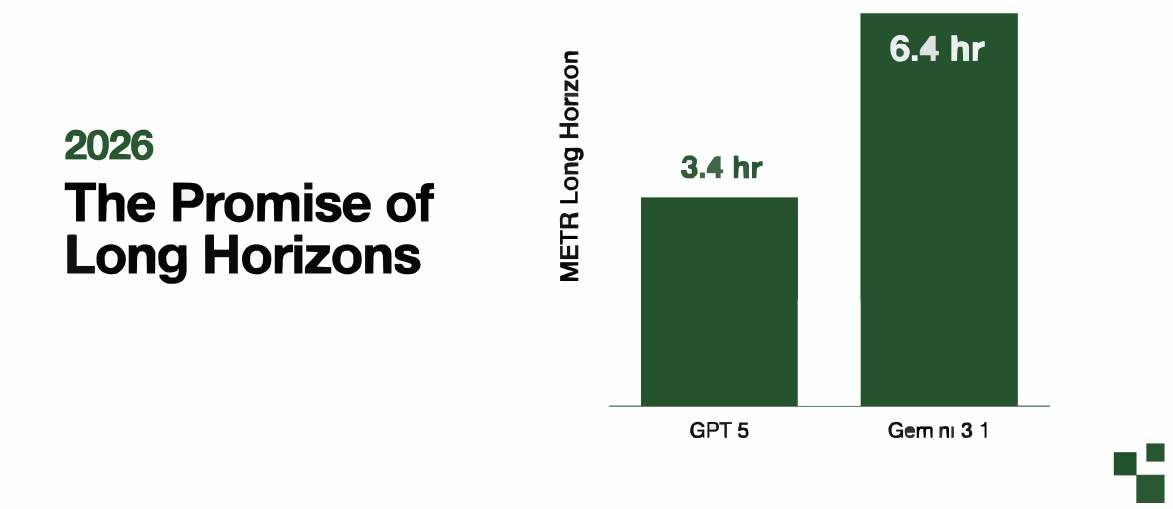


Figure 23: 2026: The Promise of Long Horizons. Bar chart of METR long-horizon task lengths: GPT-5 at 3.4 hours and Gemini 3.1 at 6.4 hours.

I've shown you 2024. Let's go to 2026.

We've moved these context fixes into the harness. What do we want next? We want agentic models that can drive our harnesses.

Everybody started targeting long-horizon agentic models. You've seen the METR reports. GPT-5 could handle tasks around 3.4 hours on that measure; Gemini 3.1 got up to 6.4 hours. That's with a harness. The models have gotten much better at using it, and that's how they're able to run for so long.

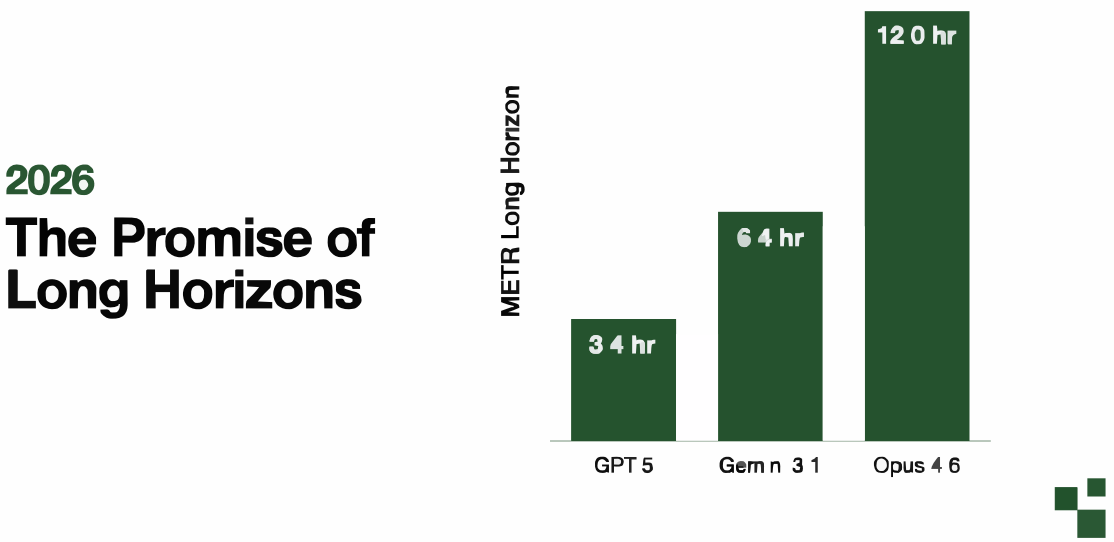


Figure 24: 2026: The Promise of Long Horizons. Bar chart of METR long-horizon task lengths: GPT-5 at 3.4 hours, Gemini 3.1 at 6.4 hours, and Opus 4.6 at 12.0 hours.

Opus 4.6 got up to around 12 hours. And now we're talking about agents running for days, swarms of agents, and models that drive harnesses over much longer periods.

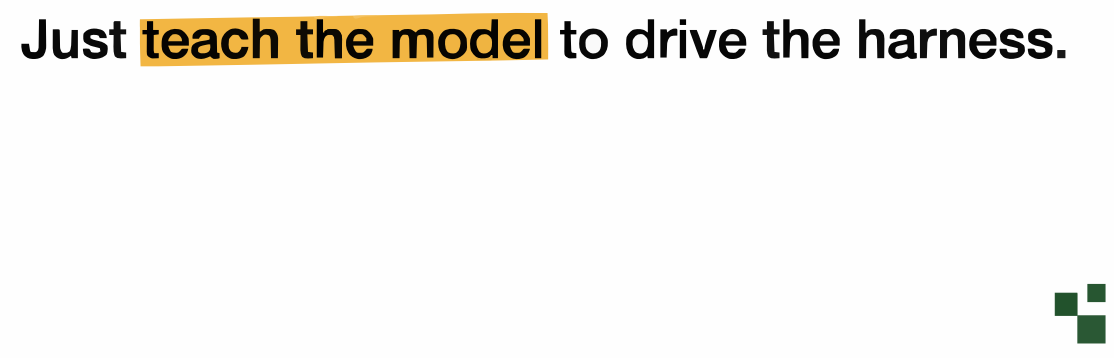


Figure 25: Just teach the model to drive the harness.

Before, the pitch was: make the context so large that we can fit everything in it.

Now the pitch is: teach the model to drive the harness and manage its own context. That's become

the dominant pattern. That's what model builders are working towards.

What new failure modes does this approach create?

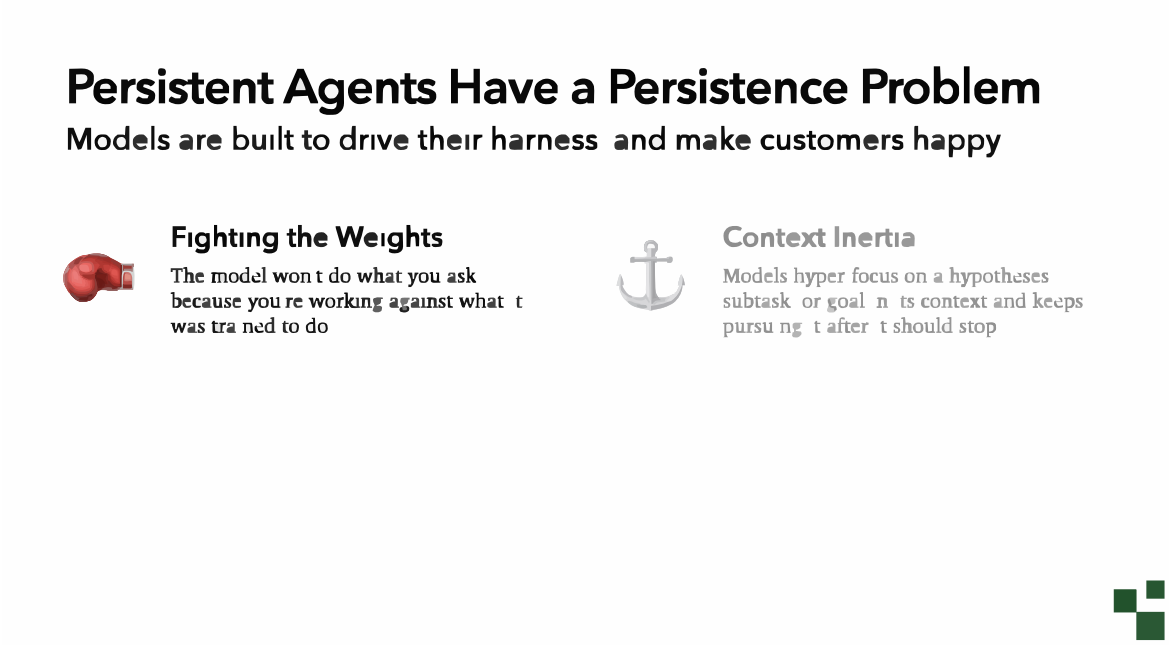


Figure 26: Persistent Agents Have a Persistence Problem: models are built to drive their harness and make customers happy. Two failure modes, with Fighting the Weights highlighted: the model won't do what you ask because you're working against what it was trained to do. Context Inertia is grayed out.

So far we've found two new failure modes.

When you train models to use harnesses really well, you end up overfitting them to a particular set of tools and environments they were trained within.

How many of you have used the Playwright MCP server, or something similar? This is the MCP conference, so probably quite a few.

Smaller models, something like Haiku or Luna, will often botch this tool call on the first attempt, then recover and adjust the parameters. The environment it was trained in had similar tools that looked different than Playwright. Whatever Anthropic or OpenAI used in training. And you have to fight against these biases.

I call this **fighting the weights**: when the model doesn't do what you ask because you're working against what it was trained to do.

It's a real problem, and it's not just for us. The labs and the people building their products have to contend with it too.

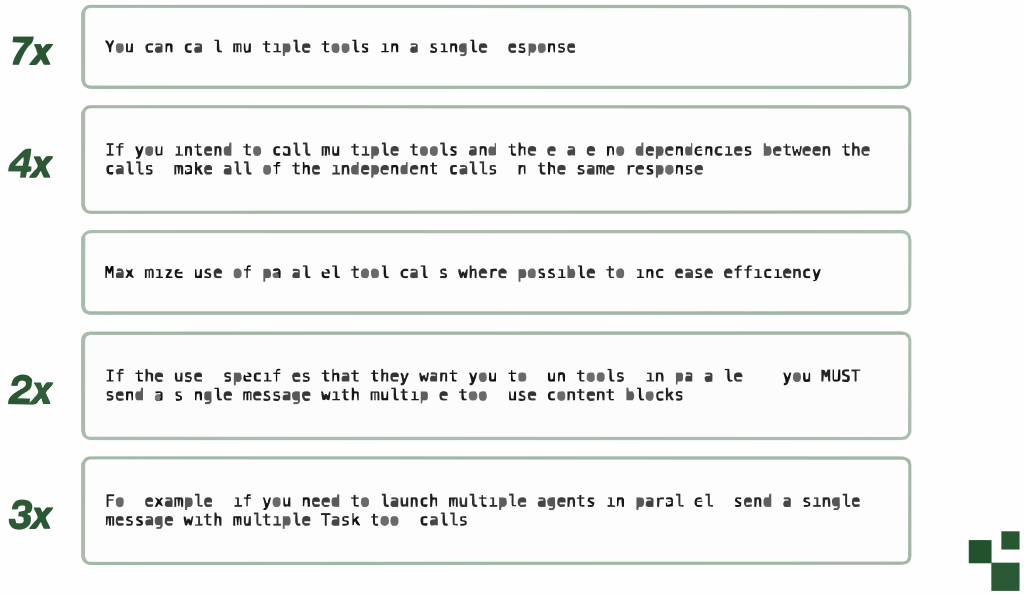


Figure 27: Excerpts from Claude Code's system prompt and tool instructions, each repeated several times: You can call multiple tools in a single response (7x); if there are no dependencies between calls, make all independent calls in the same response (4x); maximize use of parallel tool calls; if the user asks to run tools in parallel, you MUST send a single message with multiple tool use blocks (2x); to launch multiple agents in parallel, send a single message with multiple Task calls (3x).

Here's a piece of the Claude Code system prompt and some of its tool instructions.

The training environments these models learned to use were, for a long time, largely serial. They're becoming more parallel, but when I pulled these examples, their biases were towards serial behaviors. So Claude Code's instructions repeatedly say: no, no, no, we want you to use parallelism. We want you to make multiple tool calls at the same time.

The instructions repeat this over and over—seven times here, four times there, two times, three times: *please* make tool calls in parallel when they're independent.

When you crack open a prompt and see repetition and all-caps, there's a good chance you're fighting the weights. You're trying to overcome something the model was trained to do.

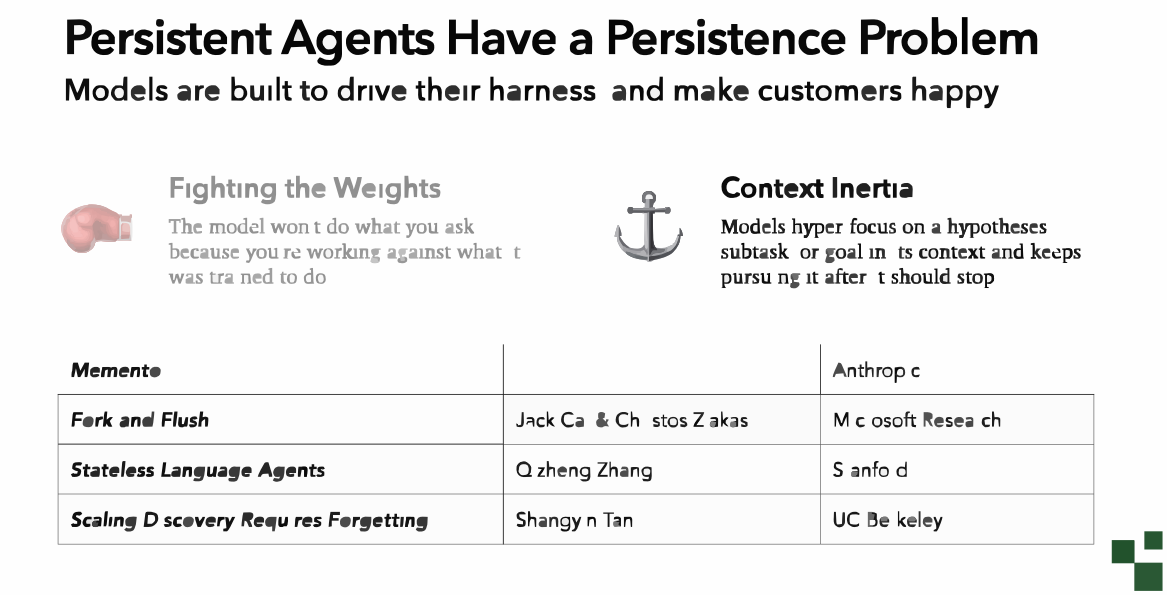


Figure 28: Persistent Agents Have a Persistence Problem, with Context Inertia highlighted: models hyper-focus on a hypothesis, subtask, or goal in their context and keep pursuing it after they should stop. A table of related work: Memento from Anthropic; Fork-and-Flush by Jack Cai and Christos Ziakas at Microsoft Research; Stateless Language Agents by Qizheng Zhang at Stanford; and Scaling Discovery Requires Forgetting by Shangyin Tan at UC Berkeley.

The next problem is something we've only started to document recently. In fact, four parties landed on it just last week.

I call it **context inertia**: models hyper-focus on a hypothesis, subtask, or goal in their context, and keep pursuing it after they should stop.

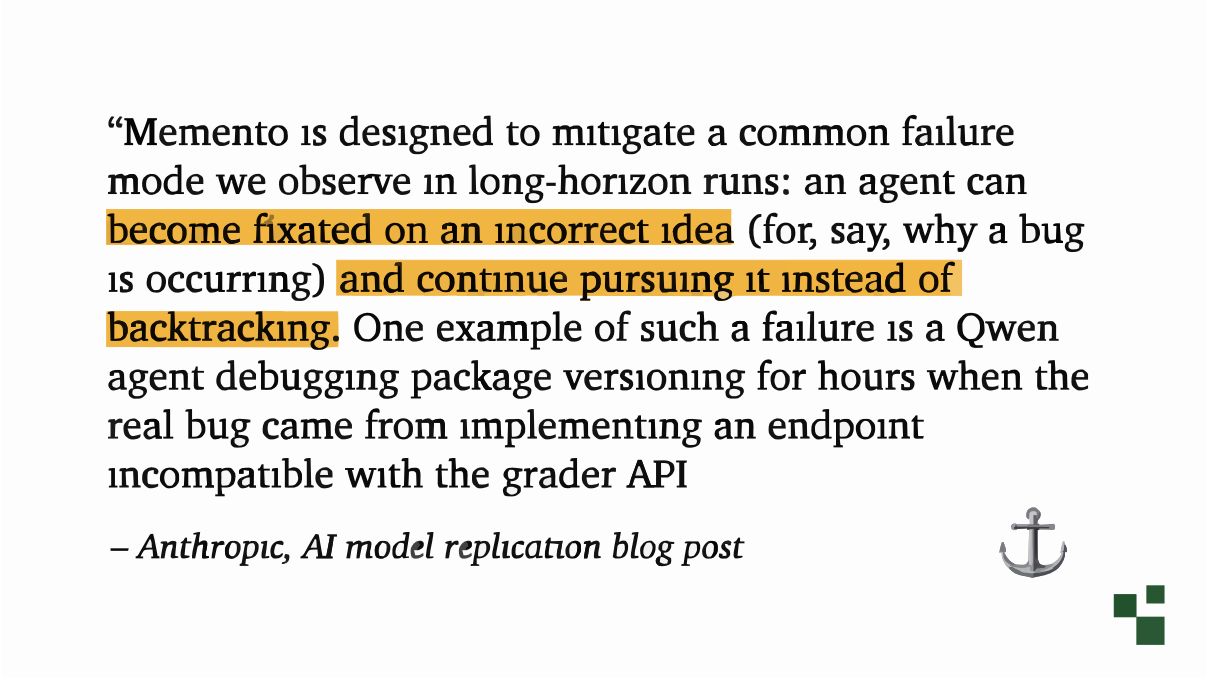


Figure 29: Quote from Anthropic's AI model replication blog post: Memento is designed to mitigate a common failure mode we observe in long-horizon runs: an agent can become fixated on an incorrect idea and continue pursuing it instead of backtracking. One example is a Qwen agent debugging package versioning for hours when the real bug came from implementing an endpoint incompatible with the grader API.

The first example was from Anthropic. In their recent post about whether models are getting close to replicating and deploying themselves onto other machines, Anthropic wrote about a harness it used for these tests called **Memento**.

What they found was that if the agents ran too long, they'd become fixated on one subtask or one proposed solution and wouldn't stop. They'd keep trying the same thing.

Their fix is delightfully simple. Every hour and a half, the agent writes down a short note, then its context is cleared. The environment is preserved. All the work it's done is still there. But the running conversation is gone, and it has to start again from a fresh perspective.

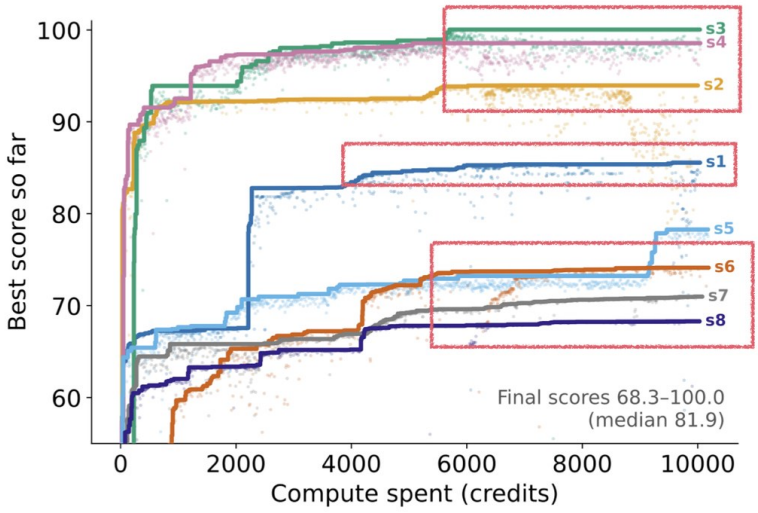


Figure 30: Chart from Fork-and-Flush by Cai and Ziakas: best score so far versus compute spent for eight agent runs. Scores jump early and then flatten into long plateaus, with final scores from 68.3 to 100.

Another paper, from Microsoft Research, is called *Fork-and-Flush*.

If you've run autoresearch experiments, you've probably seen charts like this. You make a big gain, then the line goes flat. You make another big gain, then it's flat again. That's compute, money, and time going by with very little improvement.

When the researchers looked closely, the agents weren't doing literally nothing. They were making tinier and tinier changes. They had found an idea and were hyper-focused on it, rather than searching for more creative alternatives.

In a way, they become obsessed with their own ideas. We've trained them to be persistent and focused on a goal, because that's what makes a good coding agent. But the same quality can work against them.

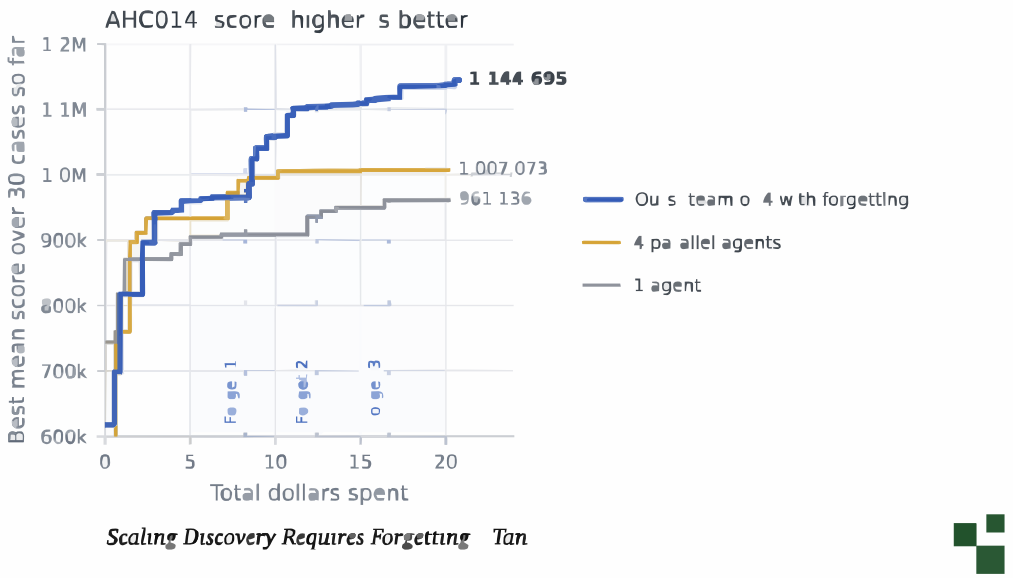


Figure 31: Chart of best mean score versus total dollars spent on the AHC014 task. A team of 4 agents with forgetting, marked by three forget points, keeps climbing to 1,144,695, beating 4 parallel agents at 1,007,073 and a single agent at 961,136. The slide caption credits Fork-and-Flush.

Here's another paper, *Scaling Discovery Requires Forgetting*, led by Shangyin Tan at UC Berkeley.

The researchers came up with essentially the same idea: periodically clear the context while keeping the artifacts produced in the environment.

The blue line is the version with forgetting. Every time it clears the context, you see another jump in performance: forget, improve; forget, improve.

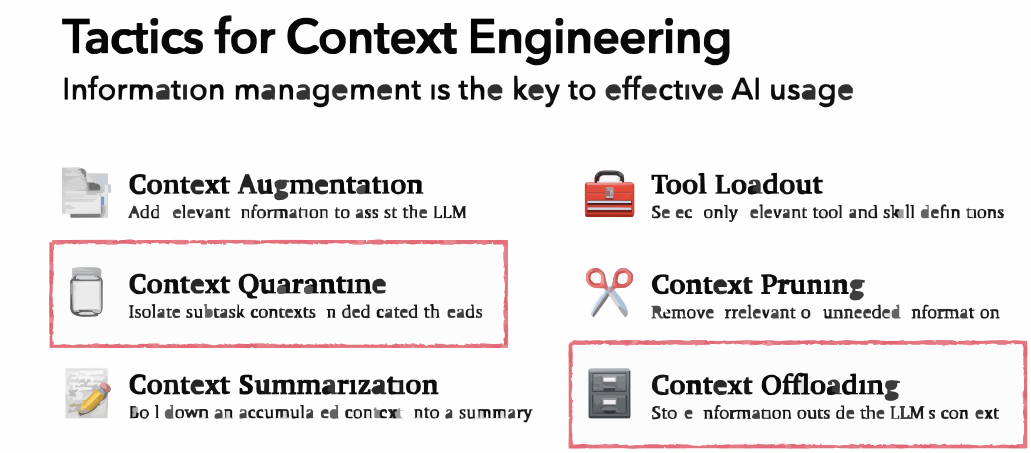


Figure 32: Tactics for Context Engineering: the six tactics again, with Context Quarantine and Context Offloading highlighted.

It's amazing that we have these brand-new models trained for increasingly complex, persistent work, and we're still using the same basic tools. We haven't discovered an entirely new context fix. We're using quarantine, offloading, and deleting the accumulated history to deal with context inertia.

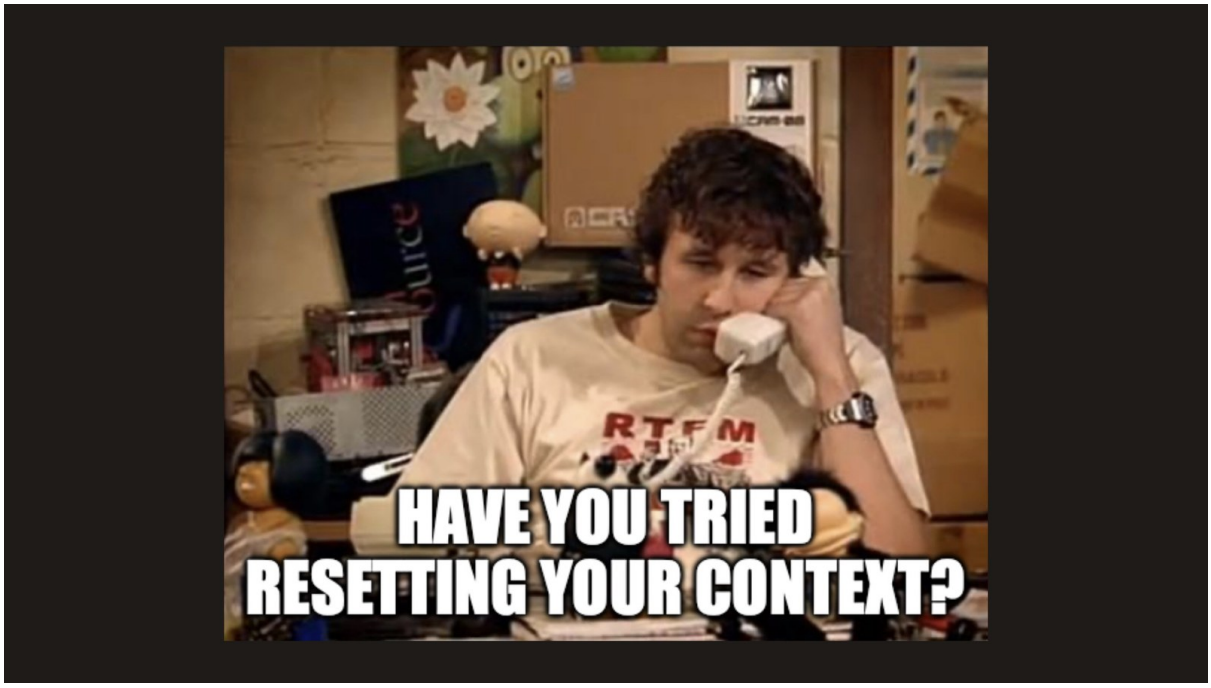


Figure 33: Meme from The IT Crowd: Roy on the phone, captioned Have you tried resetting your context?

This is the advice I keep giving now. Someone says, "My coding agent's doing something weird.

Claude's been nerfed." And I ask, "How big is your context?"

"Oh, I've got 800,000 tokens."

Delete your context.

I'm not joking. Around 200,000 tokens is my warning point.

Have you tried resetting your context?

Sometimes that really is the fix.

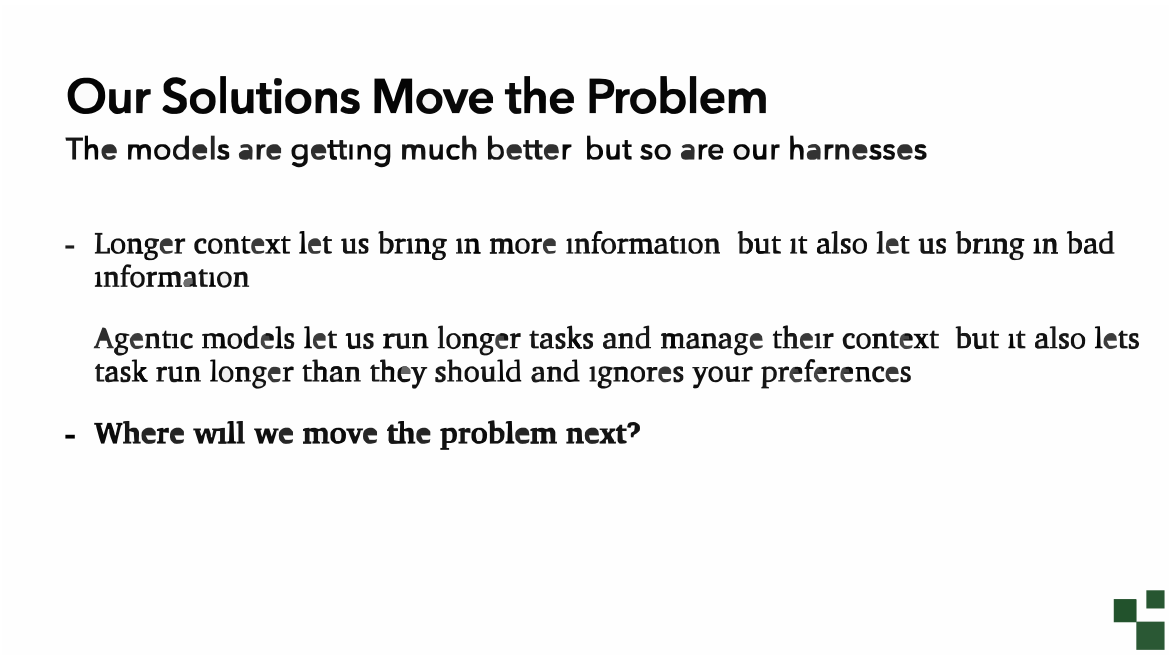


Figure 34: Our Solutions Move the Problem: the models are getting much better, but so are our harnesses. Longer context let us bring in more information, but also bad information. Agentic models let us run longer tasks and manage their context, but also let tasks run longer than they should and ignore your preferences. Where will we move the problem next?

The solutions we build into models create tomorrow's problems. The models are getting better, but so are our harnesses.

Longer context windows let us bring in more information, but they also let us bring in bad information. We had to figure out how to manage that, and we still get it wrong.

Agentic models let us run longer tasks and manage their context, but they also let tasks run longer than they should and ignore our preferences, because they were trained for their own harnesses.

So where will the problem move next?



Figure 35: 2027: The Promise of ???

What's the big promise for 2027? What's the next thing models are going to offer that creates a new set of problems for everyone in this room to deal with?

(Unless you're training your own models. But this is a context engineering session, so probably not.)

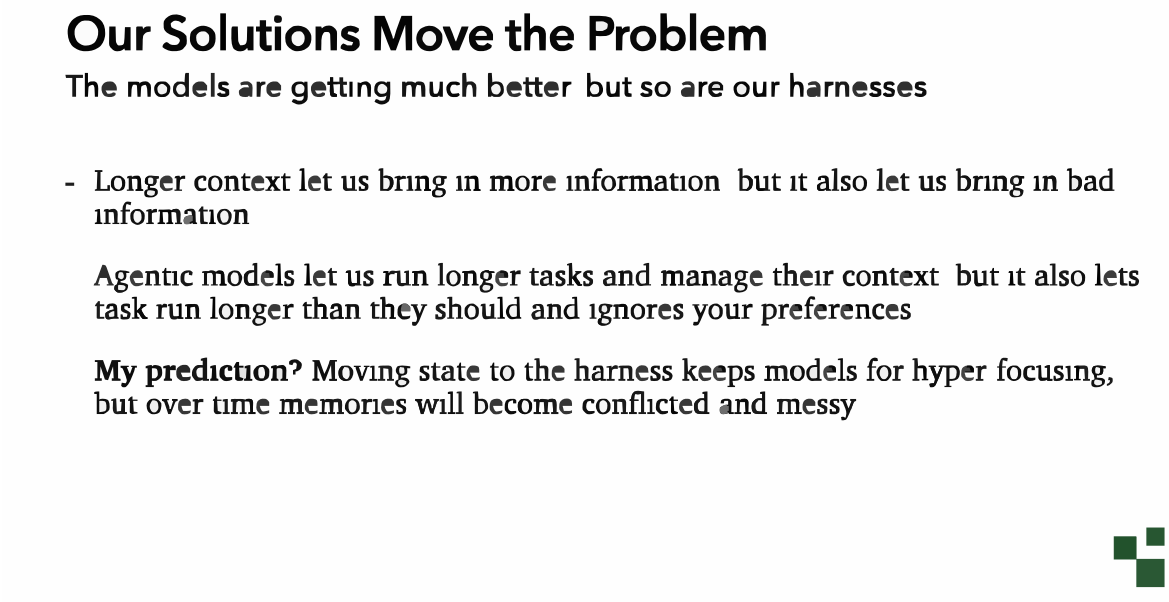


Figure 36: Our Solutions Move the Problem, with a prediction: moving state to the harness keeps models from hyper-focusing, but over time memories will become conflicted and messy.

Here's my prediction.

Think about all the personal agents: Hermes, OpenClaw, Dot, Bot, Muse, and similar systems. Very few people have really experienced these harnesses running for more than six months.

I've been running a Hermes agent for six months. When you look at the skills it creates and updates, you start to discover that the solutions to your problems can get worse over time. The skills accumulate all these weird fixes. They're general agents: they solve one task one way, then rediscover a different way to solve something similar. The instructions and memories start to build up and conflict with one another.

As we focus more on memory, we also need to think about strategic forgetting. Over time, overly precise memories become conflicted and messy, and nobody has yet figured out how to handle this.

I think that's the context problem of tomorrow.

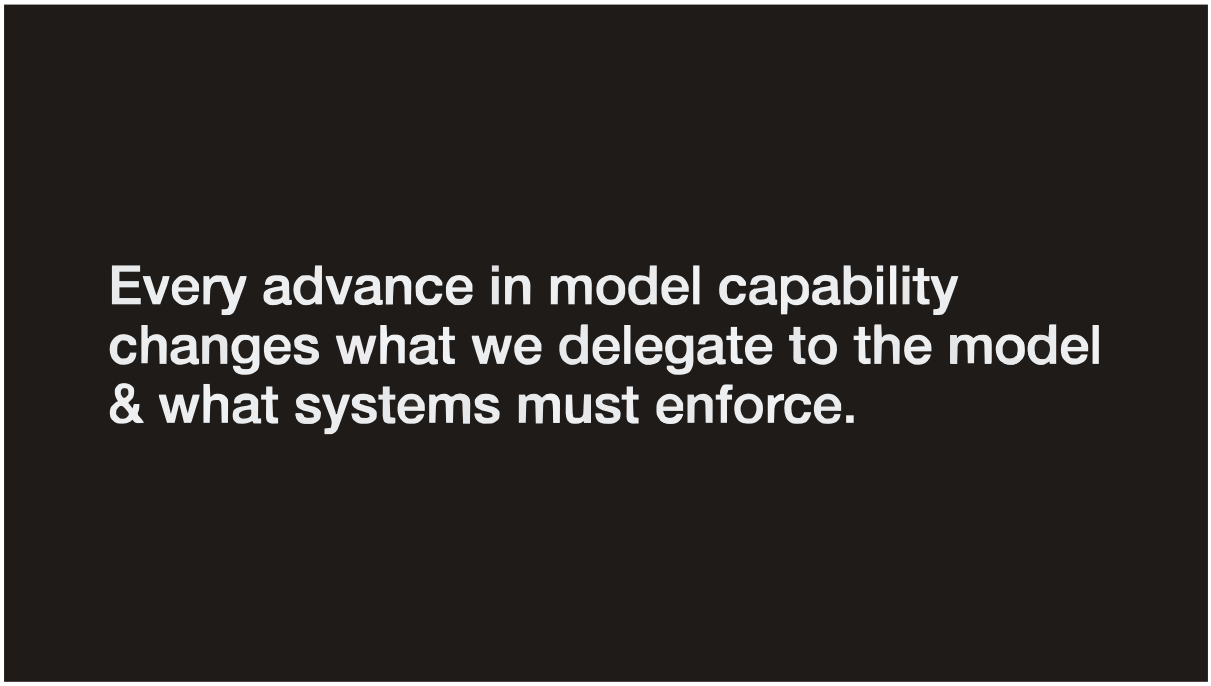


Figure 37: Every advance in model capability changes what we delegate to the model & what systems must enforce.

Every advance in model capability changes what we delegate to the model and what systems must enforce.

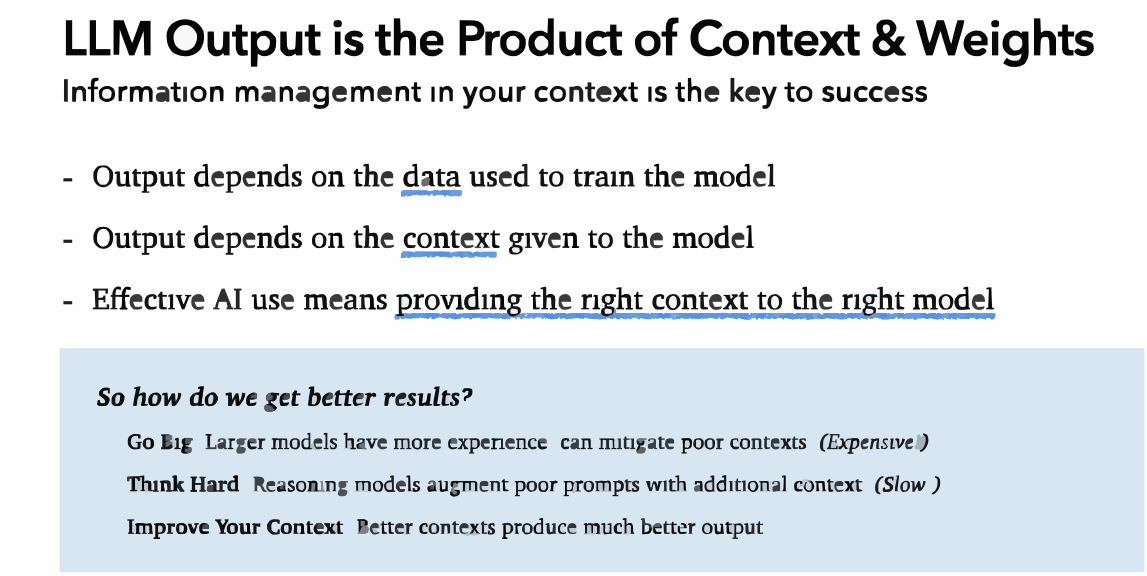


Figure 38: LLM Output is the Product of Context & Weights: output depends on the data used to train the model and the context given to the model, so effective AI use means providing the right context to the right model. To get better results: Go Big, larger models can mitigate poor contexts (expensive); Think Hard, reasoning models augment poor prompts (slow); or Improve Your Context, better contexts produce much better output.

I made this slide for a deck for training enterprise teams, about a year and a half ago, and it remains true.

Our output is the product of the context and the weights.

Our output depends on the data used to train the model. Understand it. Read the technical reports. Look at what the model was trained on and whether that's relevant to your task. That helps you build a mental model for debugging when something goes wrong.

And our output depends on the context given to the model. That's it.

So our goal is to give the right context to the right model. Either by our own discrete decisions or by the software, the harnesses, we design.

You can use bigger models, think harder, or improve your context. One of those is cheap and fast.

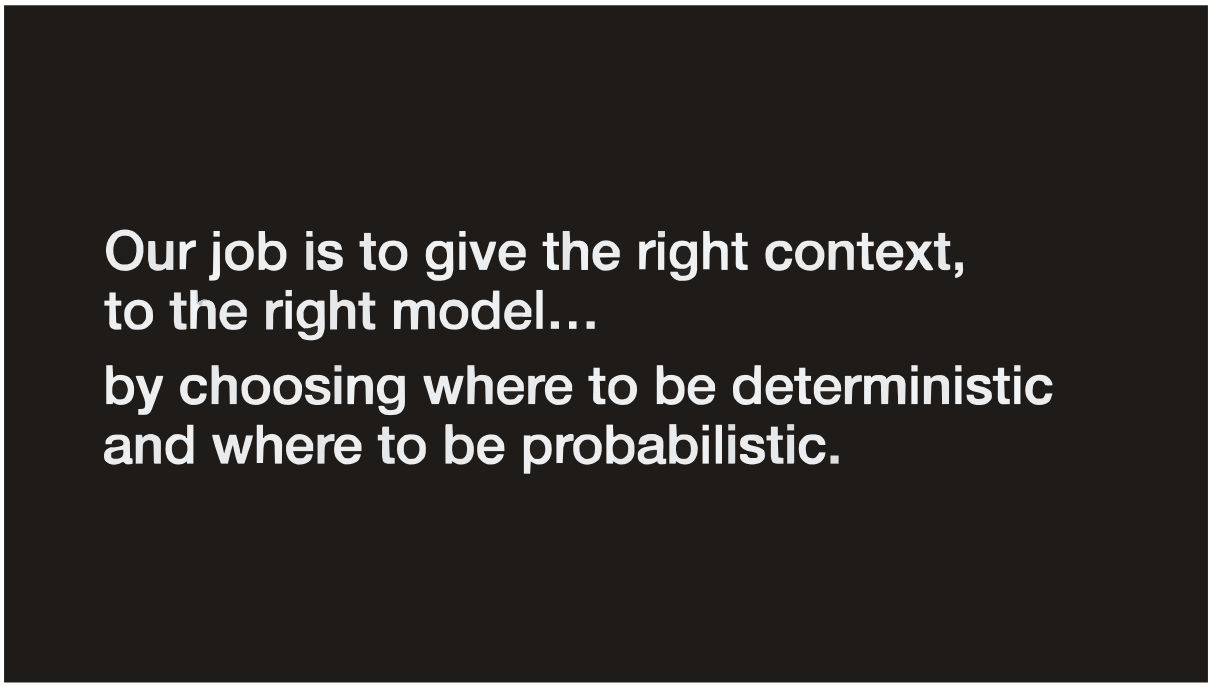


Figure 39: Our job is to give the right context, to the right model... by choosing where to be deterministic and where to be probabilistic.

When building intelligent applications, harnesses, and workflows, our job is to choose when to be deterministic (what to put in code) and when to be probabilistic (when to let the model drive).

Originally published at https://www.dbreunig.com/2026/10/24/the-state-of-context-engineering.html on 24 October 2026. Slides by Drew Breunig (Cmpnd); text reproduced here for reading offline. Slide images were vectorized from the original JPEGs.